

User Manual

This document provides concise instructions for installing, configuring, and operating NeuroScan: AI-Based Brain Stroke Prediction System, enabling both technical and non-technical users to efficiently set up and use the system.

1. System Requirements

1.1 Hardware Requirements:

Minimum:

- Processor: Intel i5 / AMD equivalent
- RAM: 8 GB
- Storage: 5 GB free disk space

Recommended:

- Processor: Intel i7 / Ryzen 7
- RAM: 16 GB (optimized for real-time ML model inference and data processing)
- Storage: SSD for faster database queries and model loading.

Peripherals:

- Fire-Boltt, Noise, or any smartwatch supporting Health Connect or Google Fit for real-time vitals synchronization.

1.2 Software Requirements:

- OS: Windows 10/11 (Recommended), Linux, or macOS.
- Environment: Python 3.10.x or higher.
- Database: SQLite (Default using app.db file) with support for SQLAlchemy ORM.
- APIs: Google Fit API / Health Connect Android API (for wearable data integration).

2. Required Python Libraries

To ensure all modules (Prediction, Wearable Sync, and Explainability) function correctly, the following dependencies must be installed:

2.1 Core Web Framework

- Flask: Main backend web framework.
- Flask-SQLAlchemy: Database toolkit and Object-Relational Mapper.
- Flask-CORS: (Critical) Enables the Wearable Data Server ([server.py](#)) to communicate with the Web Dashboard across different ports.
- Werkzeug: WSGI utility library for secure request handling.

2.2 Data Science & Machine Learning

- NumPy & Pandas: For complex data manipulation and preprocessing of vitals.
- Scikit-learn / XGBoost: The primary engines for stroke risk classification.
- Joblib: To load the [pre-trained .pkl](#) or .joblib model files.

2.3 Explainability & Testing

- SHAP / LIME: Used by the [test_explainability.py](#) module to provide transparency on why the AI predicted a specific risk level.
- Requests: Required for API testing and fetching data from the health sync server.

2.4 Data Visualization

- Chart.js: Utilized in [temp_index.html](#) to render real-time health charts (Heart Rate, SpO2, etc.).
- Matplotlib: For generating static analytical reports.

Installation Command:

```
pip install flask flask-sqlalchemy flask-cors numpy pandas scikit-learn xgboost joblib  
shap requests matplotlib
```

3. Installation and Setup

Step 1: Extract Project

- Unzip the project folder.
- Verify Files: [app.py](#), [server.py](#), [requirements.txt](#), [app.db](#), [templates/](#), and [static/](#).

Step 2: Environment Setup

Create virtual environment: [python -m venv venv](#)

Activate (Windows): [venv\Scripts\activate](#)

Activate (Linux/macOS): source [venv/bin/activate](#)

Step 3: Install Dependencies

[pip install -r requirements.txt](#)

Step 4: Initialize Database (Optional)

Skip if app.db is already present.

[python init_db.py](#)

Step 5: Launch Servers (Dual-Terminal Setup)

- **Terminal 1 (Wearable Sync API):**

[python server.py](#)

(Syncs data from smartwatch/mobile app)

- **Terminal 2 (Main Web App):**

[python app.py](#)

(Runs the User Dashboard and AI Model)

Step 6: Access System

URL: <http://127.0.0.1:5000>

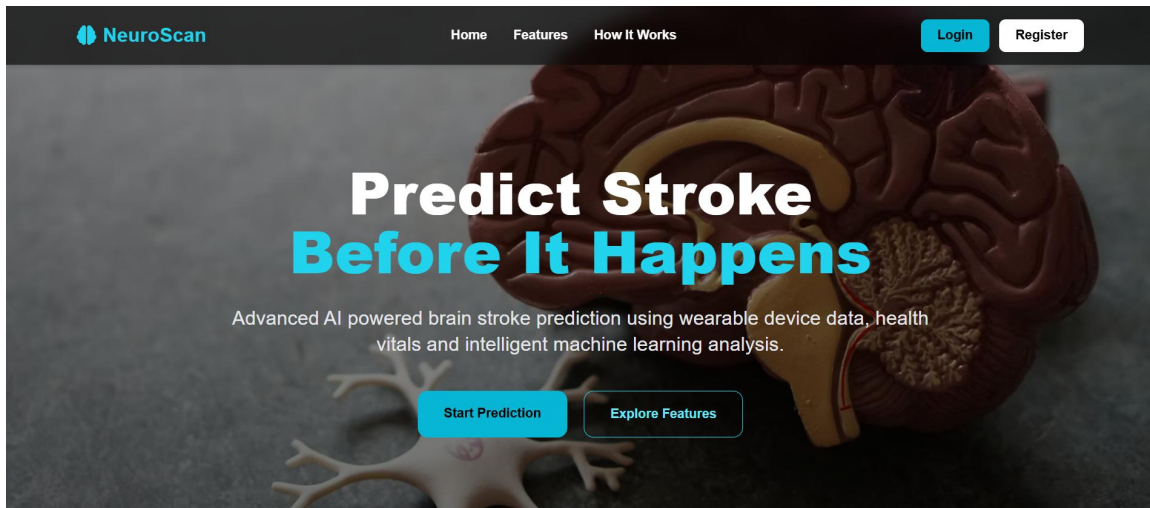
4. System Usage

This section provides a comprehensive guide to operating the **NeuroScan** system. It outlines the step-by-step workflow for users to monitor their health vitals and obtain AI-driven stroke risk predictions.

4.1 Home Page Overview

Upon launching the application, the home page provides a central navigation hub with the following options:

- Home: Overview of the system's purpose.
- Login / Register: Secure access to personal health data.



4.2 User Registration & Login

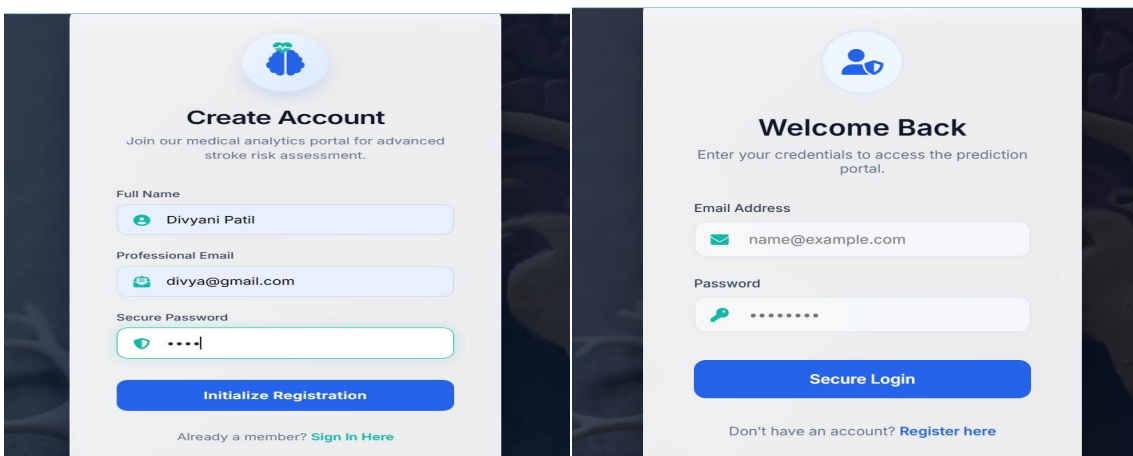
Step 1: Click on "**Register**" to create a new account.

Step 2: Enter your Full Name, Email, and a secure Password.

Step 3: Once registered, navigate to the **Login** page and enter your credentials.

System Behavior: * The system authenticates the user and establishes a secure session.

- Existing health records are retrieved from the **app.db** database.



4.3 Stroke Risk Prediction

Step 1: Navigate to the "**Start Prediction**" page.

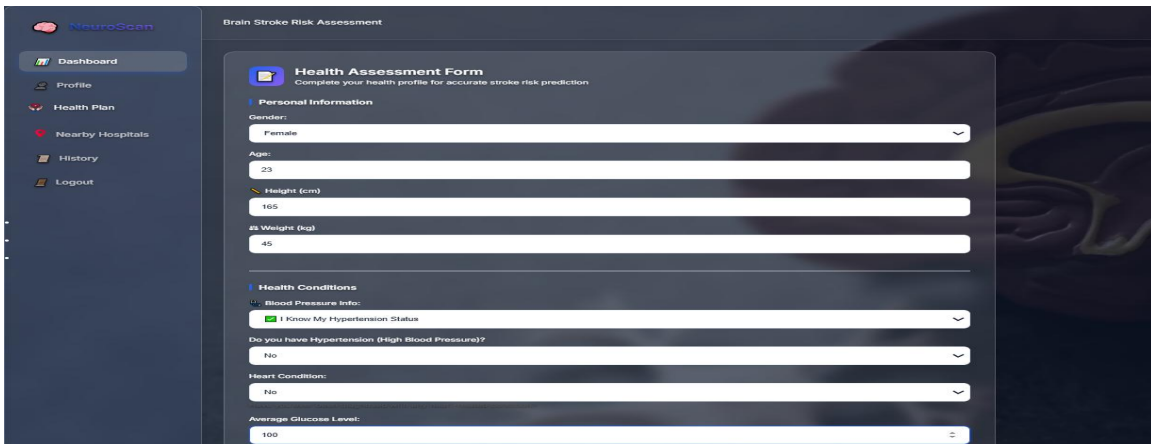
Step 2: Enter the required health details:

- Medical History: Hypertension, Heart Disease.
- Physical Metrics: Average Glucose Level, BMI.
- Lifestyle: Smoking Status, Work Type, and Residence Type.

Step 3: Click the "**Predict**" button.

System Behavior:

- The AI model processes the input alongside synced wearable data.
- The system generates a risk percentage and classifies it as Low, Moderate, or High.



The screenshot displays the NeuroScan Brain Stroke Risk Assessment interface. On the left is a sidebar with navigation links: Dashboard, Profile, Health Plan, Nearby Hospitals, History, and Logout. The main content area is titled 'Brain Stroke Risk Assessment' and contains a 'Health Assessment Form'. The form has two main sections: 'Personal Information' and 'Health Conditions'. Under 'Personal Information', there are four input fields: Gender (set to 'Female'), Age (set to '23'), Height (cm) (set to '165'), and Weight (kg) (set to '45'). Under 'Health Conditions', there are four input fields: Blood Pressure Info (set to 'I know My Hypertension Status'), Do you have Hypertension (High Blood Pressure)? (set to 'No'), Heart Conditions (set to 'No'), and Average Glucose Level (set to '100').

4.4 Wearable Data Integration

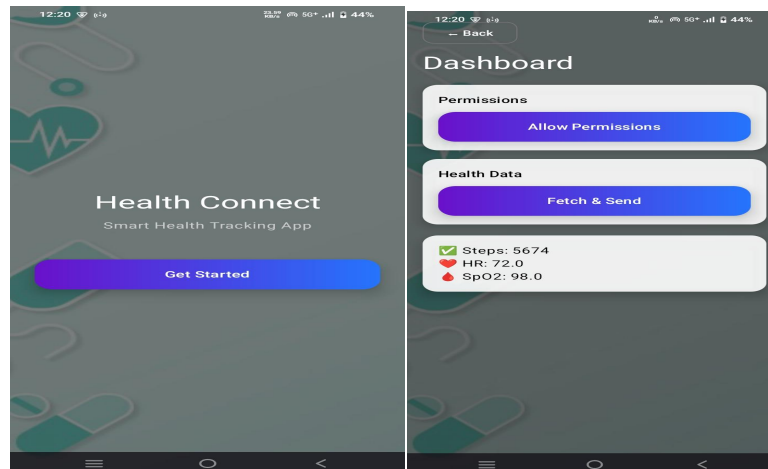
Step 1: Ensure the Wearable Sync Server ([server.py](#)) is running.

Step 2: Open the Dashboard to view live metrics synced from your smartwatch.

Data Displayed:

- Heart Rate (BPM): Real-time pulse monitoring.
- SpO2: Blood oxygen saturation levels.
- Activity: Live step count and sleep duration.

Outcome: These live metrics are automatically factored into the prediction engine for higher accuracy.



4.5 Alcohol Unit Calculator

Step 1: Access the Alcohol Unit Calculator if your specific intake units are unknown.

Step 2: Input the drink size (ml) and Alcohol by Volume (ABV%).

Step 3: Click Calculate.

System Behavior:

- The system converts the consumption into standard units.
- This value is then used to refine the "Alcohol Intake" risk factor in the prediction model.



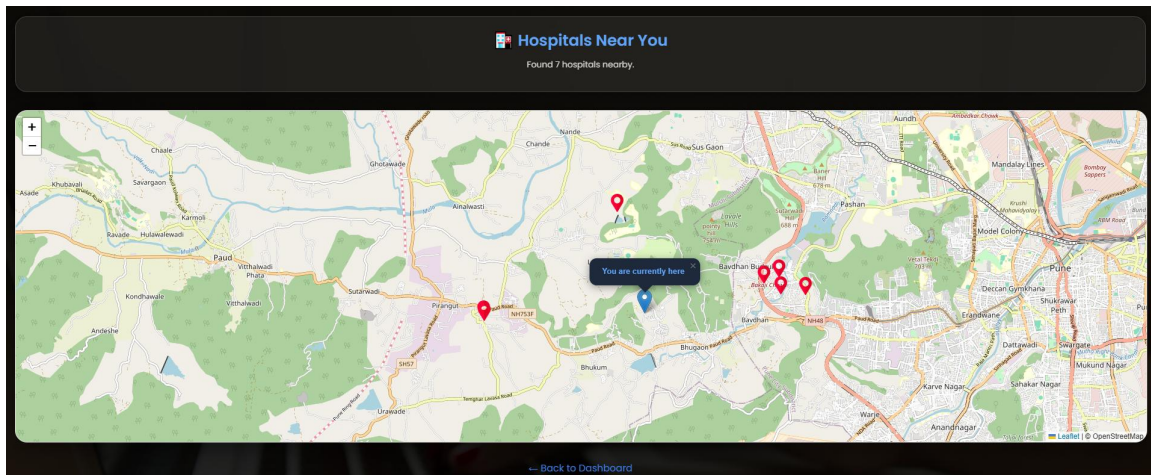
4.6 Nearby Hospitals

Step 1: Click on the "Nearby Hospitals" tab.

Step 2: Grant the browser Location Permission when prompted.

Outcome: * The system identifies your current GPS coordinates

- An interactive map displays the closest medical facilities equipped for stroke emergencies.



4.7 Health Maintenance Reports

After a prediction, the system generates a Personalized Health Plan:

- **Explainability:** Provides a breakdown of which factors (e.g., high BMI or age) increased the risk.
- **Actionable Tips:** Suggestions for diet, exercise, and yoga based on specific risk factors.
- **Resource Links:** Direct buttons to curated health videos for immediate lifestyle improvement.



5. Prediction Inputs

- Gender, Age, Height, Weight, Hypertension, Heart Disease, Average Glucose Level, Smoking Status, Alcohol Intake, Sleep Hours, Physical Activity, Family Stroke History.

6. Prediction Results

- Low Risk, Moderate Risk, High Risk with health suggestions.

7. File Structure

To ensure the AI models and wearable data sync function correctly, maintain the following directory structure:

- app/:** Core logic including the **explainability_engine.py** for AI risk breakdown.
- database/:** Contains app.db (SQLite) for storing user credentials and health history.
- models/:** Stores pre-trained ML files (**model.pkl**) used for stroke prediction.
- static/:** Images, Videos.
- templates/:** HTML UI pages (Login, Dashboard, Prediction, Health Tips).
- app.py:** Main Flask application and web routing.
- server.py:** Dedicated API server for wearable device data synchronization.

- [requirements.txt](#): List of all required Python libraries.

8. Security Guidelines

- Strong Authentication: Use complex passwords for all User and Admin accounts.
- Environment Security: Never share your .env file or Flask SECRET_KEY publicly.
- Access Control: Restrict server.py (Wearable API) access to authorized devices only.
- Database Safety: Set strict folder permissions for the app.db (SQLite) file to prevent unauthorized physical access.

9. Troubleshooting

Issue	Resolution
Python not recognized	Add Python to your system's Environment Variables (PATH).
Database Error	Verify app.db integrity or reset using python init_db.py .
Wearable Data missing	Ensure server.py is running and Port 5001 is not blocked.
Hospital Map Error	Enable "Location Permissions" in your web browser settings.
Model Loading Error	Check if model.pkl is present inside the models/ directory.

10. Maintenance

- Regular Backups: Create weekly copies of [app.db](#) in a secure external location.
- Dependency Updates: Periodically run [pip install --upgrade -r requirements.txt](#).
- Log Monitoring: Track terminal error codes (404, 500) to identify and fix bugs.
- System Cleanup: Monthly delete temporary reports and images from [static/uploads/](#).

Thank You